Tree

A few points:

So far in this course, we have discussed that a data structure has number of components. Each data structure is a type – called Abstract Data Type and each data structure supports a set of operations meaning for that data structure.

Each component knows about itself very strongly. It is cohesive. It does not know in which data structure it will be put.

Each data structure knows little about the component it contains. Whenever it requires any services from the component, it will delegate the function to that of the component. It does not probe inside the component to see what it contains. So, the data structure is also cohesive.

There is not much of dependency between the data structure and the component. They are not coupled.

By this way we realize one of the requirements of software development – high cohesion and low coupling.

We have discussed a few data structures like list, stack, queue, doubly linked list, circular queue. The components of all these data structures are in a line – they are linear. Even though the doubly linked list has two pointers in each node, they still are in a line. These data structures are called linear data structures.

There are many situations which cannot be captured in a line.

What If we trace father of a person, father's father and so on?

From Ramayana,

Lava's father is Rama. Rama father's is Dasharatha.

This is a linear relationship as a person has only one father.

What if we trace the relationship of children.

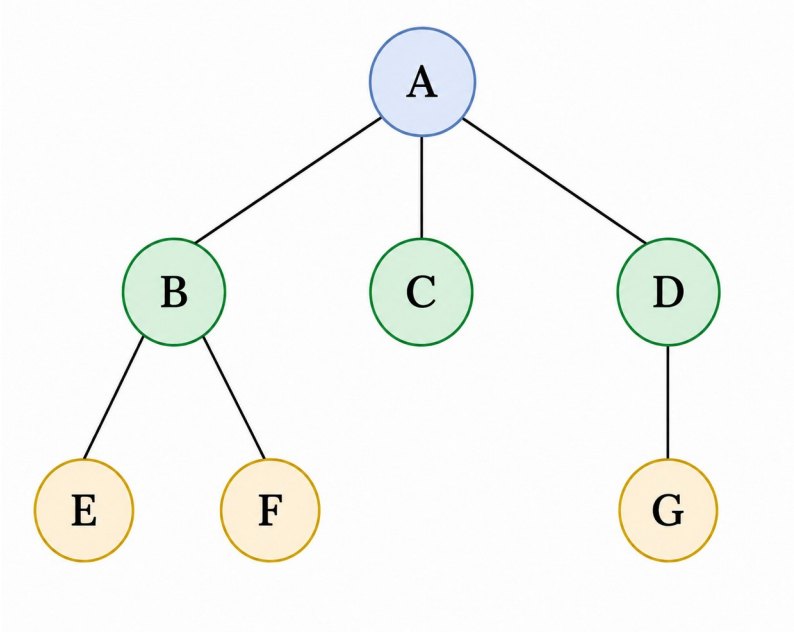
Dasharatha has 4 children. Each of them have their own children. Clearly this will not be in linear.

We do require non-linear data structures which allows one to many relatioinship.

In this course, we will study trees and graphs.

Tree:

A **tree** is a non-linear data structure used to represent hierarchical relationships. Unlike arrays, stacks, queues, and linked lists, where elements are arranged sequentially, a tree organizes elements in a parent–child structure.



This is an example of a tree.

Terminology from genealogy:

We say that A is the parent. B, C, D are children of A. They may be unordered or ordered left to right. B has two children E and F. C has no children. D has a single child G.

We say that A is the grand parent of E, F and G. We say that E, F, G are grand children of A.

B, C, D are siblings – children of the same parent.

A is an ancestor of B and also E. B and E are descendants of A.

C is the uncle of E – sibling of the parent. If there is ordering, then then C is the right uncle of E!

Sometimes these get coloured. We then talk about red uncle and black uncle.

We avoid terms indicating gender- no daughter, no mother. But so far, we are not able to find a gender-neutral term for the sibling of the parent!

Terminology from Botany:

All are referred to as nodes.

We say that A is the root. E, F, G are leaves. They are also called external nodes.

A, B, C, D are called internal nodes. The link between a parent and its child is called a branch.

A path is a sequence of nodes connected by edges.

A tree is recursively defined.

It is either empty Or

It has a root followed by number of trees – these trees are called subtree.

Summary of Terminologies:

A tree consists of a set of nodes connected by edges.

Root: the topmost node of the tree.

Parent: a node that has one or more children.

Child: a node directly below another node.

Siblings: nodes having the same parent.

Leaf / External node: a node with no children.

Internal node: a node having at least one child.

Edge: connection between a parent and a child.

Path: sequence of nodes connected by edges.

Ancestor: a node occurring on the path from the root to a given node.

Descendant: a node occurring below a given node.

Subtree: a tree formed by a node together with all its descendants.

A few more terminologies:

- The **degree of a node** is the number of children it has.

In the example above:

degree(A) = 3

degree(B) = 2

degree(C) = 0

degree(D) = 1

The **degree of a tree** is the maximum degree of any node in the tree.

Therefore, the degree of the above tree is 3.

- **Depth and level**

The **depth of a node** is the number of edges from the root to that node.

Depth(A) = 0

Depth(B) = 1

Depth(E) = 2

Nodes having the same depth are said to be at the same **level**.

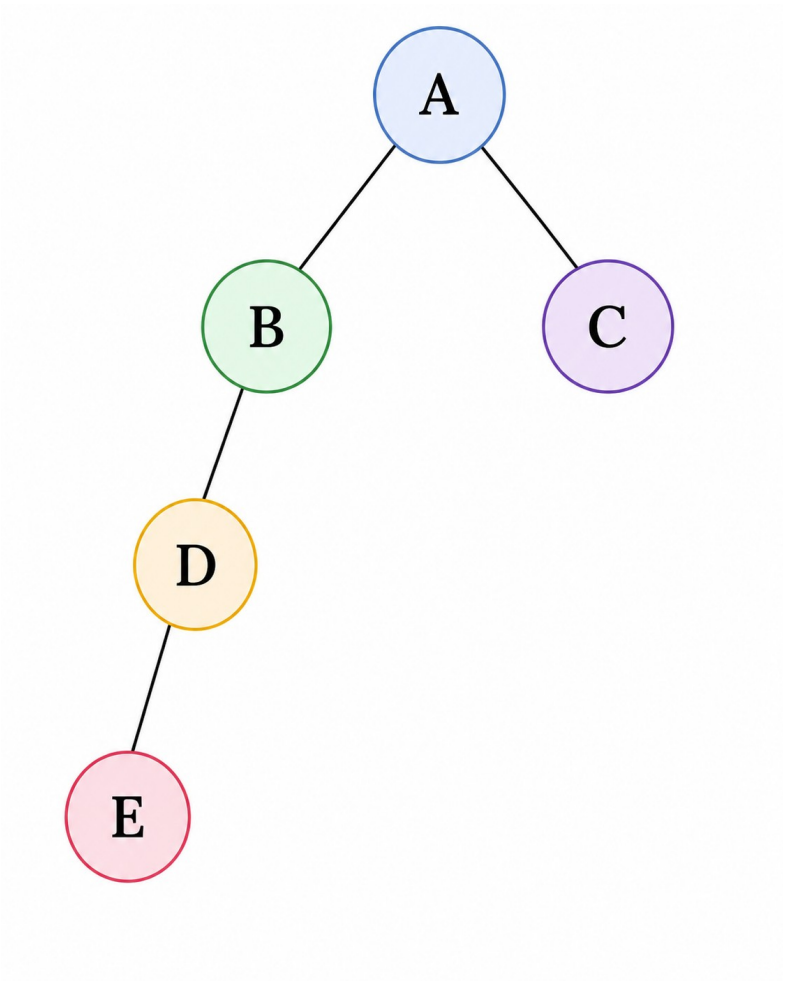
- **Height**

The **height of a node** is the length of the longest path from that node to a leaf.

A leaf has height 0.

The **height of a tree** is the height of its root.

For example:



The longest root-to-leaf path is:

A → B → D → E

It contains 3 edges. Therefore,

Height of tree = 3

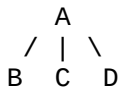
Important properties of trees:

- Acyclic : Can not start from a node, traverse and come back to the same node
- Connected. Any two nodes are connected by a path which may go through the root
- A tree of n nodes has n – 1 edges – neither more not less.
- Add an edge, we will have a cycle. Remove an edge, we will have more than one tree.

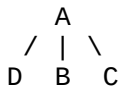
Ordered and unordered trees

In an **ordered tree**, the order of the children matters.

For example,



is considered different from:



if the tree is ordered.

In an **unordered tree**, only the parent–child relationships matter.

Forest

A **forest** is a collection of disjoint trees.

Removing the root of a tree produces a forest containing the subtrees rooted at the children of the original root.

A table indicating all the properties of the tree shown above.

Node information

Node	Parent	Children	Degree	Type
A	—	B, C, D	3	Internal
B	A	E, F	2	Internal
C	A	—	0	Leaf
D	A	G	1	Internal

Node	Parent	Children	Degree	Type
E	B	—	0	Leaf
F	B	—	0	Leaf
G	D	—	0	Leaf

Important properties

- **Number of nodes:** 7
- **Number of edges:** 6
- **Root:** A
- **Leaves:** C, E, F, G
- **Internal nodes:** A, B, D
- **Degree of tree:** 3
- **Height:** 3 edges
- **Depth of A:** 0
- **Depth of B, C, D:** 1
- **Depth of E, F, G:** 2

A possible ADT for a general tree :

- Create a tree
- Destroy the tree
- Check empty
- Check full
- Add a node
- Remove a node
- Traverse a tree
- Find root
- Find children
- Check whether leaf
- Find height of the tree
- Find the path from the root to a given node

…

In this course, we will discuss at length, a variation of trees called binary tree.

Application of Trees:

- **File System Structure:**
  This is hierarchical. A folder or a directory can have folders and directories within them apart from other files.
- **Search Trees:**
  To make searching efficient when the data is in memory, these trees are used.
- **Databases, indexing and B-trees:**
  To make searching more efficient when the data cannot be fit in memory
- **Expression Evaluation:**
  We discussed conversion of infix to postfix and evaluation of expression in postfix. Expression tree is another way of evaluation of expressions.
- **Compiler design:**
  While parsing a program, the compiler creates a tree called abstract syntax tree.
- **Document representation**
  Both XML and JSON allow for nesting forming effectively tree structure
- **Heap Tree and priority queue:**
  This is the most efficient way of implementing a priority queue
- **Process structure:**
  Processes in an operating system have relationship of the parent and the child.
- Organization structure
- Decision Trees in AI
  …

Binary Tree:

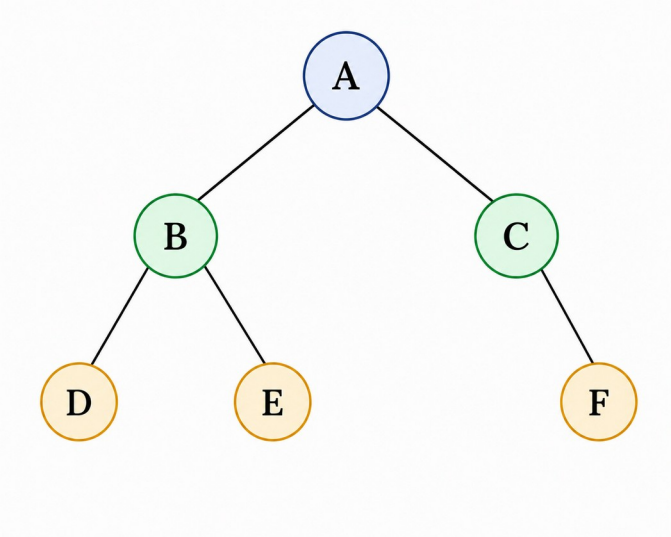
A **binary tree** is a tree in which every node has **at most two children**. The two children are distinguished as the **left child** and **right child**.

A binary tree is either:

- An **empty tree**, or
- A root node together with two disjoint binary trees called its:
    - **left subtree**
    - **right subtree**

Please observe the recursive nature of this definition. As the data structure is inherently recursive, writing recursive functions to support the interface is easy.

An example :



- A is the root.
- B is the left child of A.
- C is the right child of A.
- D and E are children of B.
- F is the right child of C.
- D, E, and F are leaves.

Binary Tree : Abstract Data Type

These are possible operations on a binary tree.

Create()
Create an empty tree

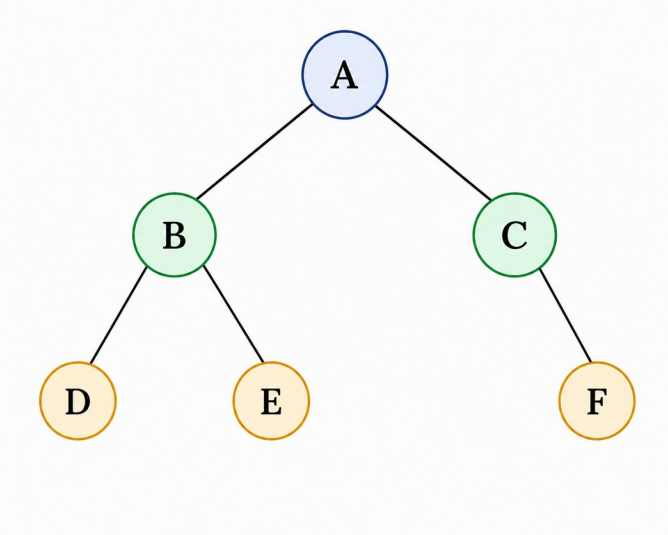
Destroy()
Remove the tree
root()
Return the root of the tree.
parent(p)
Return the parent of node p.
left(p)
Return the left child of p.
right(p)
Return the right child of p.
isLeaf(p)
Test whether p is a leaf.
component(p)
Return the component stored at p.
addRoot(x)
Create a root containing x.
addLeft(p, x)
Add x as the left child of p.
addRight(p, x)
Add x as the right child of p.
remove(p)
Remove node p and its subtree.
size()
Return the number of nodes.
Is_empty()
Test whether the tree is empty.
Is_full()

Check whether a component can be added
height()
Return the height of the tree.

We may implement a few of these interfaces in the next sections.

Inorder traversal of a binary tree:

We can visit all the components in an array or a list from one end to another. It is not possible to do so in case of a tree. One of the ways of visiting all the components in a binary tree is to visit the left subtree somehow and then the root and then the right subtree somehow. If the tree is empty, there is nothing to visit. As the left and the right subtrees are trees, we could apply this logic recursively.



Before visiting A, we visit left sub tree of A – that is the subtree B.

Before visiting B, we visit left sub tree of B – that is the subtree D.

Left sub tree of D is empty. So **visit D**.

Right sub tree of D is empty. So the sub tree D has been visited.

Left sub tree of B has been visited. So visit B.

Now visit the right sub tree of B.

Next we **visit E.**

Now, the left sub tree of A is visited. So **visit A** and then its right subtree.

This results in **visit C** and then **visit F.**

So the traversal of this tree will result in :

D B E A C F

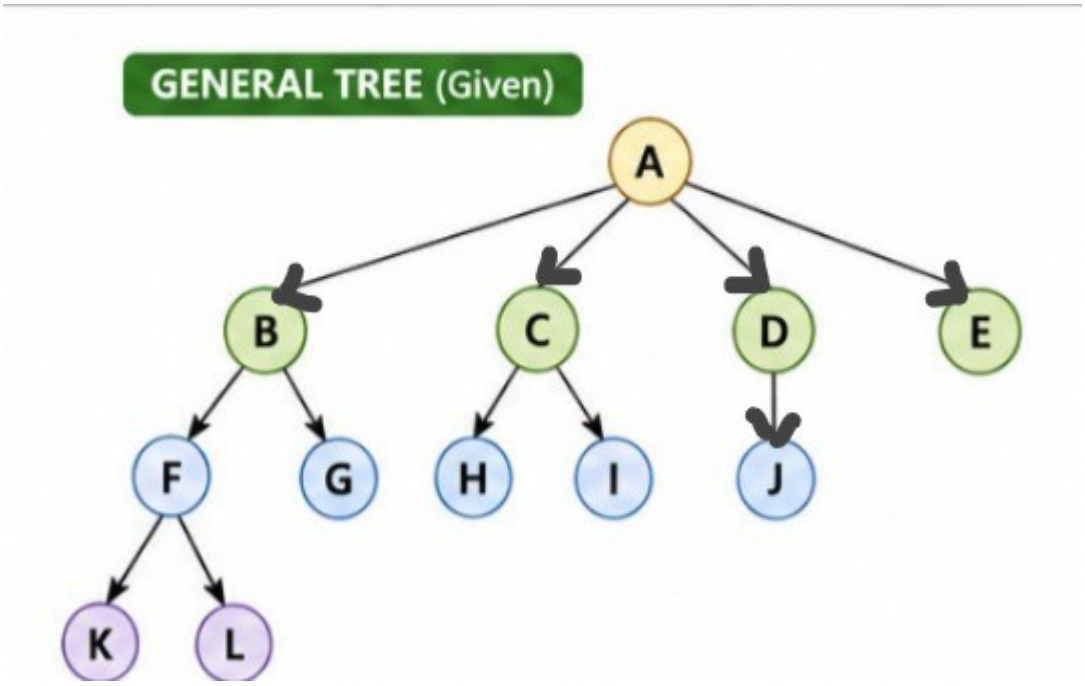
Conversion of any tree to a binary tree:

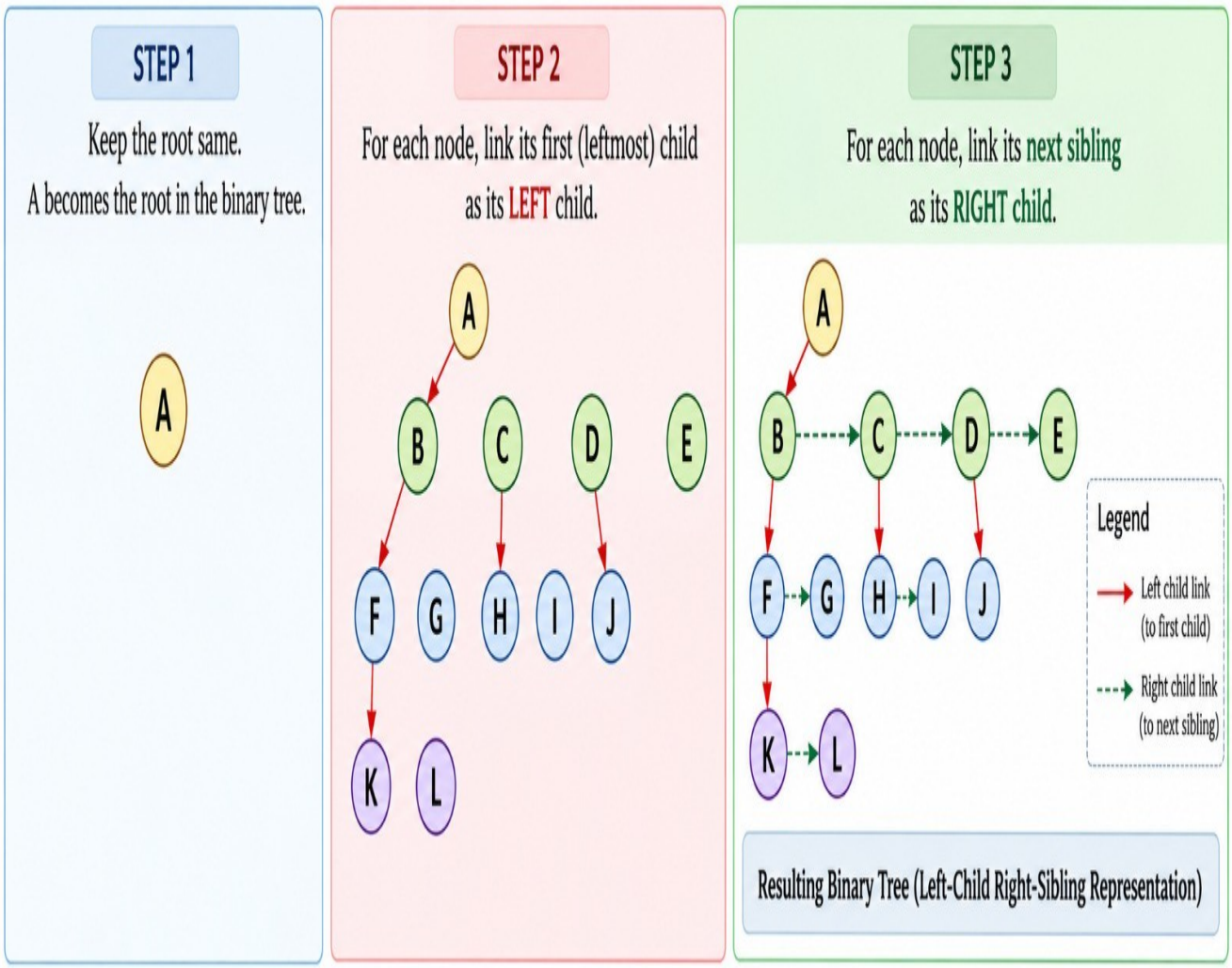
A general tree might have nodes with any number of children. Processing such a tree is difficult. But, if we can represent the general tree as a binary tree, then processing becomes easy.

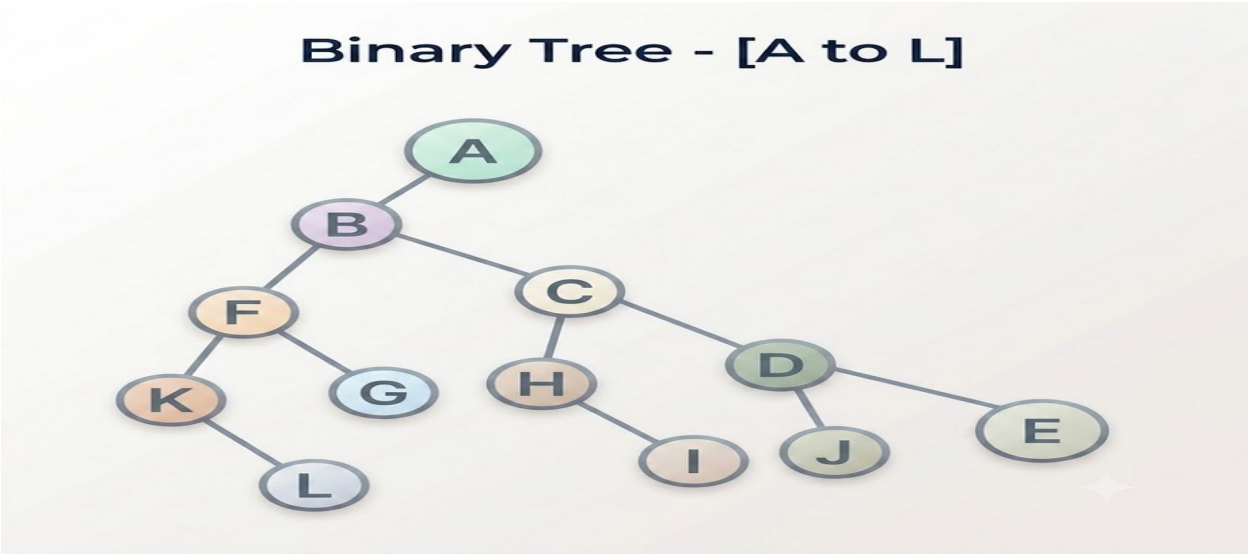
A binary tree can represent only two properties – one for the left branch and one for the right branch. If we can match this to two properties of a general tree, then we can represent any general tree as a binary tree.

This is the way we do it.

- Root of general tree will become the root of the binary tree
- Child of the node in a general tree becomes the left child of that node in the binary tree
- Sibling a node in a general tree becomes the right child of that node or attached to the right path.




STEP 1
Keep the root same.
A becomes the root in the binary tree.
A
STEP 2
For each node, link its first (leftmost) child as its LEFT child.
A
B
C
D
E
F
G
H
I
J
K
L
STEP 3
For each node, link its next sibling as its RIGHT child.
A
B
C
D
E
F
G
H
I
J
K
L
Legend
Left child link (to first child)
Right child link (to next sibling)
Resulting Binary Tree (Left-Child Right-Sibling Representation)


Binary Tree - [A to L]
A
B
C
F
D
K
G
H
E
I
J
L

Implementation of Binary Search Tree:

To start with, we will make a simple example of binary trees. We will use a special binary tree called binary search tree.

In a binary search tree, every component has a key. The components are arranged based on the key.

If the tree is empty, the component is put at the root.

Otherwise, if the key in the component at the root is higher than the key in the new component, we will insert in the left subtree.

Otherwise, we will in the right subtree.

You will observe that the insertion always happens as a leaf.

One of the implementations will be as follows.

The tree has a root. This like a list having a head.

The root points to a node structure.

The node structure has a component – called key – in this example.

Apart from that, it has two pointers – which point to the left and the right sub trees.

```
struct tree
{
        node_t *root_;
};
typedef struct tree tree_t;

struct node
{
        int key_;
        struct node *left_;
        struct node *right_;
};
typedef struct node node_t;
```

This is the simplest interface to start experimenting.

void init(tree_t *ptr_tree);

- Ground the root.

void deinit(tree_t *ptr_tree);

- Walk through the tree
- Remove the left subtree
- Remove the right subtree
- Remove the root

void disp(tree_t *ptr_tree);

- If the tree is not empty
    - Display the left subtree
    - Display the root
    - Display the right subtree

void insert(tree_t *ptr_tree, int key);

- If the tree is empty make this the root
- Otherwise compare the keys
- Decide to insert in the left subtree or the right subtree

We mentioned that the tree is inherently a recursive data structure.

But the tree structure by itself is not recursive – the node structure which is self-referential is recursive. So, the functions in the tree call their corresponding recursive counter part in nodes.

Let us have a look at them.

Traversal:

Observe that the tree function disp calls disp_recursive of the node.

The disp_function of the node calls itself on its left subtree, displays the root and

Calls itself on its right subtree.

```
void disp(tree_t *ptr_tree)
{
	printf("Tree : ");
	disp_recursive(ptr_tree->root_);
	printf("\n");
}

static void disp_recursive(node_t* temp)
{
	if(temp)
	{
		disp_recursive(temp->left_);
		printf("%d ", temp->key_);
		disp_recursive(temp->right_);
	}
}
```

Insert:

```
void insert(tree_t *ptr_tree, int key)
{
        node_t *temp = (node_t*)malloc(sizeof(node_t));
        temp->key_ = key;
        temp->left_ = temp->right_ = NULL;
        ptr_tree->root_ = insert_recursive(ptr_tree->root_, temp);
}
```

The above tree routine creates a node. Assumes that it succeeds in memory allocation. Populates the node. Calls the insert_recursive function of the node to return the root.

This function is very interesting. If the tree is empty, then this new node becomes the root. Otherwise based on the comparison of the key, we try to insert in the left subtree or the right subtree. If the insertion is in the left subtree, then left of the root is the root for that subtree. Similarly, if the insertion is in the right subtree, the right of the root is the root for that subtree. So, the return value of the function is assigned to the appropriate root.

You may want to draw and check how this works for a tree of at least 3 nodes.

```
static node_t* insert_recursive(node_t* root, node_t* temp)
{
        if(root == NULL)
        {
                root = temp;
        }
        else if(root->key_ > temp->key_)
        {
                root->left_ = insert_recursive(root->left_, temp);
        }
```

```
        else
        {
                root->right_ = insert_recursive(root->right_, temp);
        }
        return root;
}
```

Deinitialize:

Like other functions, deinit of tree calls the deinit_recursive of node.

```
void deinit(tree_t *ptr_tree)
{
        deinit_recursive(ptr_tree->root_);
        ptr_tree->root_ = NULL;
}
```

This recursive function on the node clears the left subtree recursively, then the right subtree recursively and then the root.

Please node that the order matters.

```
static void deinit_recursive(node_t* root)
{
        if(root)
        {
                deinit_recursive(root->left_);
                deinit_recursive(root->right_);
                free(root);
        }
}
```

Adding a few more functions in the interface of the tree:

Count the number of nodes

int number_of_nodes(tree_t *ptr_tree);

This function of tree calls the recursive function of the node.

The number of nodes in an empty tree is 0.

Otherwise we count the root node and add the number of nodes in the left subtree and the right subtree.

```
// tree
int number_of_nodes(tree_t *ptr_tree)
{
	return number_of_nodes_recursive(ptr_tree->root_);
}
int number_of_nodes_recursive(node_t* temp)
{
	if(temp == NULL)
	{
		return 0;
	}
	else
	{
		return 1 + number_of_nodes_recursive(temp->left_)
			+ number_of_nodes_recursive(temp->right_);
	}
}
```

Count the number of leaves

int number_of_leaves(tree_t *ptr_tree);

This function of tree calls the recursive function of the node.

The number of leaves in an empty tree is 0.

If both the left and right subtrees are empty, it is a leaf. We count it.

Otherwise we count the number of leaves in the left subtree and the number of leaves in the right subtree.

```
// node

int number_of_leaves(tree_t *ptr_tree)
{
	return number_of_leaves_recursive(ptr_tree->root_);
}
int number_of_leaves_recursive(node_t* temp)
{
	if(temp == NULL)
	{
		return 0;
	}
	else if(temp->left_ == NULL && temp->right_ == NULL)
	{
		return 1;
	}
	else
	{
		return number_of_leaves_recursive(temp->left_)
			+ number_of_leaves_recursive(temp->right_);
	}
}
```